Alat za praćenje zauzeća memorije firmwarea (fwmem)

I. kontrolna točka: priprema repozitorija i definicija problema

Mauro Pahović · Programiranje: skriptni jezici · 31.10.2026.

1. Uvod i motivacija

Mikrokontroleri u pametnim satovima, vozilima, dronovima ili kućanskim uređajima imaju vrlo ograničenu memoriju, često svega nekoliko stotina kilobajta flasha i nekoliko desetaka kilobajta RAM-a. Ako se firmware tijekom razvoja poveća preko kapaciteta čipa, više se ne može učitati na uređaj, a uzrok je često teško pronaći jer se kod mijenja postupno. Zbog toga embedded timovi nakon svakog builda prate koliko memorije program troši i što je do povećanja dovelo. Taj se posao u praksi gotovo uvijek rješava skriptama koje obrađuju izlaz alata za prevođenje i povezuju ga s verzioniranjem i automatizacijom (CI).

Ovaj projekt automatizira upravo taj tehnički proces. Tema je odabrana jer povezuje skriptiranje s embedded softverom, područjem u kojem se planira daljnji razvoj. Uključuje obradu tekstualnih tokova, povezivanje postojećih alata i automatizaciju, a to je upravo ono što skriptni jezici omogućuju.

2. Definicija problema

Prevedeni program podijeljen je u sekcije: `.text` (strojni kod), `.rodata` (konstante), `.data` (inicijalizirane globalne varijable) i `.bss` (neinicijalizirane globalne varijable). Flash otprilike zauzimaju `.text + .rodata + .data`, a RAM `.data + .bss` (uz stog i heap). Podaci su dostupni u izlazu alata `size` i `nm` te u linkerovoj `.map` datoteci, ali u sirovom obliku koji je nepregledan i ne odgovara na praktična pitanja:

- koliko je postotaka flasha i RAM-a iskorišteno,
- koje su funkcije i varijable najveće,
- koji objektni modul troši najviše,
- što je i za koliko poraslo u odnosu na prethodnu verziju,
- je li build prekoračio dopuštenu granicu.

3. Ciljevi projekta

1. Iz izlaza `size`, `nm` i `.map` datoteke izvući zauzeće sekcija, najveće simbole i zauzeće po modulima.
2. Usporediti dva builda ili dva git commita i ispisati koji su simboli novi, obrisani ili promijenjeni i za koliko bajtova.
3. Generirati izvještaj: tablice u terminalu te HTML/PNG s grafovima.
4. Provjeriti prag kapaciteta iz konfiguracijske datoteke, vratiti izlazni kod različit od nule pri prekoračenju i ugraditi provjeru u git hook ili CI workflow.
5. Alat demonstrirati na vlastitom C projektu uz različite razine optimizacije (`-O0`, `-O2`, `-Os`) te po mogućnosti na open-source firmwareu.

4. Rješenje i tehnologije

Jezgra alata napisana je u **Bashu** (parsiranje tekstualnog izlaza pomoću `awk`, `sed`, `sort`, `join` i rad s gitom), a za grafičke izvještaje koristit će se **Python** s bibliotekom matplotlib. Ulaz alata su ELF i `.map` datoteke, a izlaz tablice, izvještaji i izlazni kod. Za izradu ELF datoteka dovoljan je običan `gcc` sa zastavicama `-ffunction-sections -fdata-sections -Wl,--gc-sections -Wl,-Map`, a po potrebi `arm-none-eabi-gcc`. Fizički mikrokontroler nije potreban jer se analiziraju datoteke koje proizvodi kompajler. Razvojno okruženje je Arch Linux.

5. Organizacija repozitorija

Repozitorij `fwmem` ima sljedeću strukturu: `src/` (skripte alata), `tests/` (testovi u bats i pytest), `config/` (primjer konfiguracije s kapacitetom čipa i pragovima), `samples/demo_fw/` (mali C projekt s Makefileom koji generira ELF i `.map`) i `docs/` (izvješća). Na grani `main` bit će stabilna verzija, a nove funkcionalnosti razvijat će se na granama `feature/<naziv>` i spajati u `main`. Commit poruke bit će kratke imperativne rečenice, a svaka funkcionalnost spremat će se u zaseban commit kako bi napredak bio vidljiv u povijesti.

6. Plan rada po kontrolnim točkama

KT	Rok	Planirani sadržaj
I.	01.11.	Repozitorij, struktura, README, demo C projekt s Makefileom, definicija problema i plan
II.	27.11.	Bash skripta za sekcije i top N simbola (`size`, `nm`), parser `.map` datoteke za zauzeće po modulima, prvi testovi
III.	18.12.	Usporedba dvaju buildova/commitova, izvještaj u terminalu i HTML/PNG s grafovima, konfiguracija kapaciteta, analiza `-O0` naspram `-Os`
IV.	22.01.	Provjera praga s izlaznim kodom, git hook / GitHub Actions, testovi, dokumentacija, demo na open-source projektu, završno izvješće

7. Dosad napravljeno

Pripremljen je repozitorij s `README.md`, `.gitignore` i definiranom strukturom direktorija. Izrađen je demo C projekt od četiri modula (`main`, `uart`, `config`, `sensor`) koji sadrži kod, konstante te inicijalizirane i neinicijalizirane globalne varijable, pa se u svim sekcijama dobivaju netrivijalne vrijednosti. Pripadajući Makefile parametriziran je razinom optimizacije i izlaznim direktorijem, tako da se isti izvor može prevesti u više varijanti za usporedbu. Provjereno je da `size` i `nm` nad rezultatom daju očekivani ispis. Promjene su spremljene u najmanje dva commita na grani `main`.

8. Rizici i mjere

Format `.map` datoteke razlikuje se među verzijama linkera, pa će se parser prvo prilagoditi GNU ld-u, a odstupanja će se rješavati testnim uzorcima u `tests/`. Veći projekti poput Zephyra, koji traže dodatno

postavljanje SDK-a, ostavljaju se za posljednju kontrolnu točku, a do tada će se koristiti vlastiti demo projekt.